

Partie 1 : Présentation Générale de la Plateforme CTFd

1. Contexte et Définition d'un CTF

Dans le domaine de la cybersécurité, un **CTF (Capture The Flag)** est une compétition d'enfer permettant aux participants de mettre en pratique leurs compétences en résolution de problèmes de sécurité (hacking, cryptographie, rétro-ingénierie, etc.). Les utilisateurs doivent exploiter des vulnérabilités ou analyser des systèmes pour découvrir une chaîne de caractères secrète appelée « **Flag** » (généralement au format `flag{...}`). La soumission de ce flag sur une plateforme dédiée leur attribue des points.

2. Qu'est-ce que CTFd ?

CTFd est le framework open-source leader mondial pour l'hébergement et la gestion de compétitions Capture The Flag. Développé en Python, il fournit une infrastructure logicielle complète clé en main pour les organisateurs.

Fonctionnalités Natives Principales :

- **Gestion des utilisateurs et des équipes** : Inscription, profils individuels et regroupements de joueurs.
- **Tableau des scores (Scoreboard)** : Calcul en temps réel des classements avec graphiques d'évolution dynamiques.
- **Catalogue de défis (Challenges)** : Organisation des exercices par catégories (Web, Crypto, Forensic) avec attribution de points fixes ou dégressifs.
- **Système de validation** : Vérification automatique des flags soumis par les candidats et déblocage d'indices.

3. Architecture Technique de Base

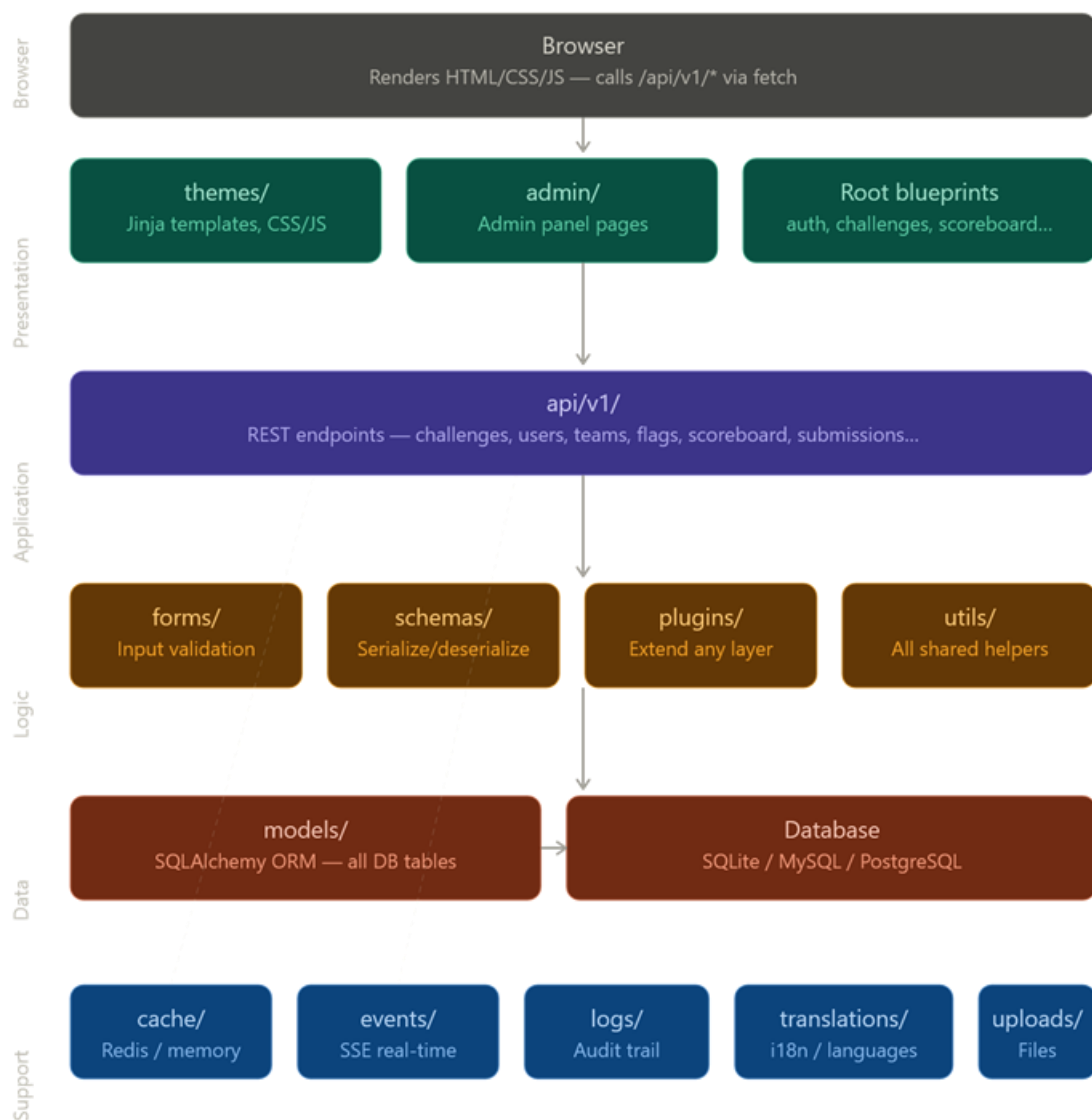
CTFd repose sur une pile technologique moderne, modulaire et hautement extensible :

- **Backend (Moteur de l'application)** : Conçu avec le framework **Flask (Python)** pour la gestion des routes, des sessions et de la logique métier.
- **Base de données (Persistance)** : Utilisation de l'ORM **SQLAlchemy** pour s'interfacer de manière transparente avec un serveur de base de données relationnelle (généralement **MySQL** ou **PostgreSQL**).
- **Frontend (Interface Utilisateur)** : Propulsé par des templates **Jinja2**, combinés avec **Bootstrap** et du JavaScript standard pour un rendu responsive et dynamique.
- **Conteneurisation (Déploiement)** : Orchestré nativement via **Docker** et **Docker Compose**, isolant l'application (ctfd), la base de données (db) et le cache (cache utilisant Redis).

4. L'Extensibilité par les Plugins

Le point fort majeur de CTFd réside dans son **architecture native orientée plugins**. Le framework permet d'injecter du code Python et des templates HTML personnalisés dans des dossiers isolés (sous CTFd/plugins/) sans jamais modifier le code source de base de l'application. C'est précisément cette flexibilité qui a été exploitée au cours de ce projet pour concevoir nos 9 modules d'administration.

5. Architecture CTFd :



Partie 2 : Suivi Logique et Chronologique des Sprints

1. Introduction à la Méthodologie de Développement

Le projet a été mené selon une approche de développement incrémentale et modulaire. Chaque fonctionnalité a été livrée de bout en bout : de la modélisation en base de données jusqu'à la création des interfaces d'administration et des moteurs d'exportation, garantissant ainsi un système directement opérationnel à la fin de chaque itération.

2. Sprint 1 : Fondations et Architecture des Données Critiques

Le premier sprint a posé l'infrastructure de persistance et les barrières de sécurité indispensables à la supervision des sessions d'examens.

◆ Issue #1 — Système Intégral de Gestion des Sponsors

1. Problématique et Objectif Fonctionnel

Dans la version native de CTFd, la mise en valeur des partenaires académiques et industriels sur la page d'accueil (Landing Page) se fait par modification statique des fichiers thématiques HTML (index.html). Cette approche impose une manipulation directe du code source par les organisateurs, augmentant le risque d'altération de la syntaxe graphique et bloquant toute modification à la volée pendant l'événement. L'objectif était de concevoir un sous-système autonome permettant d'ajouter, modifier, supprimer (CRUD) et ordonner l'affichage des sponsors directement depuis le panneau d'administration, avec une répercussion dynamique instantanée sur la page d'accueil publique.

2. Modélisation et Persistance des Données (Base de données MySQL)

Pour s'affranchir des contraintes de fichiers statiques, une table relationnelle dédiée nommée sponsors a été injectée dans l'instance MySQL à l'aide de l'ORM SQLAlchemy intégré à CTFd.

```
from CTFd.models import db

class Sponsors(db.Model):
    __tablename__ = 'sponsors'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(128), nullable=False)
    image_link = db.Column(db.Text, nullable=False)
    order = db.Column(db.Integer, default=0, nullable=False)

    def __init__(self, name, image_link, order=0):
        self.name = name
        self.image_link = image_link
        self.order = order
```

- ❑ **id (Primary Key)** : Clé primaire auto-incrémentée assurant l'unicité de chaque enregistrement.
- ❑ **name** : Chaîne de caractères stockant le nom du partenaire (clé d'identification textuelle).
- ❑ **image_link** : Champ textuel stockant l'URL absolue ou le chemin d'accès vers le composant graphique du logo (hébergé en local ou sur un CDN sécurisé).
- ❑ **order** : Entier servant d'indice d'ordonnement pour la priorité d'affichage séquentiel.

3. Logique Métier Backend (Python & Flask)

Le cœur logique repose sur l'implémentation d'un Blueprint Flask indépendant au sein du module de chargement des extensions (load(app)).

A. Traitement de la page d'accueil publique :

La route native de l'index de CTFd a été interceptée pour effectuer une requête SQL triée de manière ascendante selon l'indice de priorité :

```
sponsors_list = Sponsors.query.order_by(Sponsors.order.asc()).all()
```

Cette collection d'objets est ensuite passée directement au contexte de rendu du template d'accueil.

B. Contrôleur d'administration (CRUD et Sécurité) :

Une route d'administration dédiée /admin/sponsors a été créée, protégée impérativement par le décorateur @admins_only pour empêcher toute escalade de privilèges.

- **Create (Ajout)** : Lors d'une requête POST, le système récupère les paramètres du formulaire. Pour éviter les conflits d'indexation, le backend calcule dynamiquement la valeur maximale actuelle de la colonne order et attribue automatiquement max_order + 1 au nouveau sponsor.
- **Delete (Suppression)** : Le contrôleur intercepte l'ID du sponsor ciblé, valide son existence, exécute db.session.delete(sponsor) et valide la transaction avec db.session.commit().

4. Interface Utilisateur & Intégration Frontend (Jinja2 & HTML)

A. Rendu sur la page d'accueil :

À l'aide du moteur de rendu Jinja2, une boucle itère sur la liste transmise par le backend pour générer un conteneur responsif (Bootstrap Flexbox/Grid) :

```
<div class="sponsors-grid">
  {% for sponsor in sponsors_list %}
    <div class="sponsor-item">
      
  {% endfor %}
</div>
```

B. Le Panneau d'Administration et l'Algorithme d'Ordonnement :

L'interface d'administration affiche la liste des sponsors sous forme de tableau interactif. Chaque ligne comprend des contrôles de déplacement (Flèches Haut/Bas) connectés à des routes de traitement dédiées (/admin/sponsors/swap/<int:id>/up).

- **L'algorithme de Swap** : Lorsqu'un administrateur clique sur "Haut" pour le Sponsor **B** (ex: order = 3), le backend recherche l'élément **A** qui possède l'indice directement inférieur (ex: order = 2). Les deux valeurs de la colonne order sont inversées au sein d'une même transaction SQL.
- **Gestion des conditions aux limites (Sécurité Jinja2)** : Pour empêcher un crash serveur ou une corruption d'index lors d'une action impossible (cliquer sur "Haut" pour le premier élément de la liste ou "Bas" pour le dernier), des balises conditionnelles évaluent la position de la boucle. Si la condition loop.first ou loop.last est validée, l'élément d'action HTML passe instantanément en mode **disabled (grisé)**, bloquant de manière absolue toute interaction frontend hors-limites.

◆ Issue #2 — Suivi du Temps par Défi

1. Problématique et Objectif Fonctionnel

Dans sa configuration d'origine, CTFd fonctionne sur un modèle purement transactionnel : la plateforme enregistre la milliseconde exacte à laquelle un flag correct est soumis, mais elle n'a aucun moyen de savoir ce qui se passe avant cet événement. Cela crée un angle mort total pour les enseignants. Il est impossible de savoir combien de temps une équipe passe réellement sur un exercice avant de trouver la solution, ou si elle abandonne un sujet pour un autre.

L'objectif de cette fonctionnalité lors du Sprint 1 était de poser les bases techniques d'un **Moteur de Télémétrie Temporelle** axé sur la collaboration. Le but était de concevoir la structure de stockage backend (Data Layer), d'établir un mécanisme de calcul automatique de la durée des sessions et de mettre en place un algorithme de plafonnement pour neutraliser les temps d'inactivité des équipes.

2. Modélisation et Persistance des Données (Base de données MySQL)

Pour enregistrer l'activité temporelle des groupes sans impacter les performances d'affichage de la plateforme, une table relationnelle dédiée nommée challenge_time_track a été créée dans MySQL à l'aide de l'ORM SQLAlchemy.

```

from CTFd.models import db
import datetime

class ChallengeTimeTrack(db.Model):
    __tablename__ = 'challenge_time_track'
    id = db.Column(db.Integer, primary_key=True)
    team_id = db.Column(db.Integer, db.ForeignKey('teams.id', ondelete='CASCADE'),
        nullable=False)
    challenge_id = db.Column(db.Integer, db.ForeignKey('challenges.id',
        ondelete='CASCADE'), nullable=False)
    start_time = db.Column(db.DateTime, default=datetime.datetime.utcnow,
        nullable=False)
    end_time = db.Column(db.DateTime, nullable=True)
    duration = db.Column(db.Integer, default=0, nullable=False)

    def __init__(self, challenge_id, team_id):
        self.challenge_id = challenge_id
        self.team_id = team_id
        self.start_time = datetime.datetime.utcnow()

```

- **id (Clé Primaire)** : Entier auto-incrémenté assurant l'unicité de chaque session d'activité ouverte.
- **team_id (Clé Étrangère)** : Lié directement à la table native teams de CTFd, avec la règle de sécurité **ondelete='CASCADE'**. Si une équipe est supprimée par l'administrateur, ses données de temps sont nettoyées automatiquement pour éviter les lignes orphelines.
- **challenge_id (Clé Étrangère)** : Lié à la table native challenges pour identifier l'exercice consulté.
- **start_time & end_time** : Horodatages universels (UTC) marquant le début et la fin de la consultation du défi par l'équipe.
- **duration** : Entier calculant le delta en **secondes brutes**, format idéal pour permettre des calculs mathématiques rapides en arrière-plan.

3. Logique Métier Backend et Plafonnement d'Inactivité (Python)

La logique métier intercepte l'affichage des exercices pour alimenter l'historique de temps de la base de données.

A. Initialisation et Clôture des Sessions :

Pour garantir un suivi réaliste, le système ne se contente pas d'écraser un enregistrement existant lorsqu'une équipe navigue d'un défi à un autre. Le backend génère une **nouvelle ligne de session indépendante à chaque ouverture** de fenêtre.

- *Exemple de parcours (Défi A → Défi B → Défi A)* : Si l'équipe travaille sur le Défi A, le ferme pour analyser le Défi B, puis revient sur le Défi A, le système crée trois lignes distinctes en base de données. Les deux périodes de travail sur le Défi A forment ainsi deux segments temporels isolés (avec leurs propres [start_time et end_time]).

- *Agrégation cumulative* : Lors des calculs statistiques, le système utilise l'agrégation SQL SUM et un regroupement GROUP BY challenge_id. Cela permet d'additionner parfaitement tous les segments d'un même défi pour obtenir le temps total cumulé exact passé par l'équipe sur cet exercice.

B. Algorithme de Plafonnement de Sécurité (MAX_SESSION_WINDOW) :

Le principal défi technique de la télémétrie temporelle est le comportement des étudiants qui laissent une fenêtre de défi ouverte en arrière-plan pendant des heures sans y travailler (par exemple, pendant la pause déjeuner). Laisser ce temps s'accumuler fausserait complètement les statistiques des enseignants.

Pour résoudre ce problème, un **Mécanisme de Plafonnement Strict** a été développé au niveau du backend :

```
MAX_SESSION_WINDOW = 900

def close_team_session(track_entry):
    if not track_entry.end_time:
        track_entry.end_time = datetime.datetime.utcnow()

    delta_seconds = (track_entry.end_time - track_entry.start_time).total_seconds()

    if delta_seconds > MAX_SESSION_WINDOW:
        track_entry.duration = MAX_SESSION_WINDOW
        track_entry.end_time = track_entry.start_time +
datetime.timedelta(seconds=MAX_SESSION_WINDOW)
    else:
        track_entry.duration = int(delta_seconds)
```

Si la session d'ouverture d'un défi par une équipe dépasse les 15 minutes sans aucune interaction ou soumission, le moteur backend détecte cette anomalie d'inactivité. Il coupe le calcul de la durée réelle, affecte automatiquement la valeur maximale autorisée de 900 secondes à la colonne duration, et ferme proprement la transaction en base de données.

◆ Issue #7 — Brute-force / Flag Sharing Detection (Détection des Menaces et de la Triche)

1. Problématique et Objectif Fonctionnel

L'évaluation académique sur une plateforme de CTF standard se heurte à deux vulnérabilités majeures qui faussent les notations :

1. **Les attaques par force brute (Brute-Force)** : Des étudiants tentent de deviner les flags en envoyant des centaines de variantes par script automatisé sur les formulaires de soumission.

2. **La collusion et le partage de flags (Flag Sharing)** : Un étudiant résout un défi et partage directement la chaîne de caractères secrète (le flag) avec ses camarades, qui la valident instantanément sur leur propre compte sans avoir lu l'énoncé.

L'objectif de cette fonctionnalité lors du Sprint 1 était de concevoir un **Module de Détection et de Sanction Automatique** agissant en arrière-plan. Le système doit analyser le comportement des soumissions en temps réel pour bloquer instantanément les attaquants et lever des alertes immédiates en cas de validation suspecte.

2. Modélisation et Persistance des Données (MySQL)

Pour assurer le suivi des incidents de sécurité et permettre à l'enseignant de consulter l'historique des fraudes, une table relationnelle dédiée nommée `security_alerts` a été créée dans MySQL.

```
from CTFd.models import db
import datetime

class SecurityAlerts(db.Model):
    __tablename__ = 'security_alerts'
    id = db.Column(db.Integer, primary_key=True)
    team_id = db.Column(db.Integer, db.ForeignKey('teams.id', ondelete='CASCADE'),
        nullable=False)
    challenge_id = db.Column(db.Integer, db.ForeignKey('challenges.id',
        ondelete='CASCADE'), nullable=False)
    alert_type = db.Column(db.String(64), nullable=False)
    details = db.Column(db.Text, nullable=True)
    timestamp = db.Column(db.DateTime, default=datetime.datetime.utcnow, nullable=False)

    def __init__(self, team_id, challenge_id, alert_type, details=None):
        self.team_id = team_id
        self.challenge_id = challenge_id
        self.alert_type = alert_type
        self.details = details
```

- **id (Clé Primaire)** : Identifiant unique auto-incrémenté pour chaque alerte de sécurité enregistrée.
- **team_id & challenge_id (Clés Étrangères)** : Associent l'incident à l'équipe coupable et au défi concerné, avec une contrainte de suppression en cascade (`ondelete='CASCADE'`).
- **alert_type** : Chaîne de caractères catégorisant la menace (soit Brute-force, soit Collusion).
- **details** : Champ textuel stockant le contexte technique (ex : le nombre exact de tentatives infructueuses ou le délai de résolution suspect).

3. Logique Métier Backend et Algorithmes d'Interception (Python)

Le système déploie un middleware d'interception branché sur le cycle de vie de Flask via le décorateur d'application. Il surveille chaque requête envoyée vers l'API de soumission des flags de CTFd (`/api/v1/submissions`).

A. Algorithme Anti-Brute-Force (Machine à États et Pénalité)

Lorsqu'une équipe soumet un flag incorrect, le backend intercepte l'échec et interroge la table native submissions pour analyser l'historique récent de cette équipe sur ce défi spécifique :

1. **Calcul de la Fenêtre Glissante** : Le script compte le nombre de soumissions invalides survenues au cours des **120 dernières secondes** (2 minutes).
2. **Déclenchement du Seuil** : Si ce compteur est supérieur ou égal à **5 tentatives**, le système génère immédiatement une alerte dans la table security_alerts.
3. **Application de la Sanction** : Le backend modifie temporairement l'état d'accès de l'équipe pour ce défi en appliquant un verrouillage strict de **180 secondes** (3 minutes). Pendant cette période de pénalité, toute nouvelle soumission de l'équipe est rejetée automatiquement avec un message d'erreur, bloquant l'utilisation de scripts de force brute.

B. Algorithme Anti-Collusion (Analyse de la Vitesse de Résolution)

Pour détecter le partage direct de flags entre étudiants, le système croise les données de soumission avec notre moteur de télémétrie temporelle (challenge_time_track) développé dans l'Issue #2 :

1. **Calcul du Délai d'Assimilation** : Au moment exact où une équipe soumet un flag *correct*, le backend récupère la ligne de suivi de temps active pour ce groupe sur ce défi. Il calcule la différence entre l'heure de soumission actuelle et le start_time (le moment où la fenêtre du défi a été ouverte).
2. **Détection de la Triche** : Si cette différence est **strictement inférieure à 15 secondes**, le système considère qu'il est techniquement impossible pour un être humain de lire l'énoncé, d'analyser l'infrastructure de TP et de concevoir la solution.
3. **Enregistrement de l'Alerte** : Le défi est validé pour l'équipe (car le flag est correct), mais le backend insère instantanément une alerte de type Collusion dans la table security_alerts. Les détails enregistrent le message explicite : *"Défi validé en X secondes, suspicion forte de partage de flag"*. L'enseignant dispose ainsi d'une preuve factuelle pour sa notation.

3. Sprint 2 : Interfaces d'Administration, Analyses Pédagogiques et Reporting

Le deuxième sprint a transformé les données brutes en tableaux de bord visuels et a enrichi les fonctionnalités de gestion pour les enseignants.

◆ Évolution de l'Issue #2 — Ajout de l'Utilisateur Unique dans la table de Time Tracking

1. Problématique et Objectif Fonctionnel

Lors du Sprint 1, le moteur de télémétrie temporelle (`challenge_time_track`) collectait les données de focalisation uniquement à l'échelle de l'équipe (`team_id`). Bien que cette modélisation soit adaptée aux compétitions de type Hackathon où le score est global au groupe, elle s'est révélée insuffisante dans un contexte pédagogique universitaire.

Dans une salle de classe, les enseignants effectuent des notations individuelles et nominatives. Si trois étudiants forment une équipe, le modèle du Sprint 1 ne permettait pas de savoir quel membre était actif sur la machine et passait réellement du temps à chercher la solution. L'objectif de cette évolution au début du Sprint 2 était de **decoupler le suivi temporel** en y intégrant l'identifiant unique de l'utilisateur connecté (`user_id`), permettant ainsi une granularité analytique absolue par étudiant.

2. Migration de la Base de Données et Refonte Structurale (MySQL)

Pour implémenter cette granularité sans perdre l'historique des données du Sprint 1, la structure relationnelle de la table `challenge_time_track` a été modifiée via l'ORM SQLAlchemy pour intégrer une nouvelle contrainte de clé étrangère.

```
from CTFd.models import db
import datetime

class ChallengeTimeTrack(db.Model):
    __tablename__ = 'challenge_time_track'
    id = db.Column(db.Integer, primary_key=True)
    team_id = db.Column(db.Integer, db.ForeignKey('teams.id', ondelete='CASCADE'),
        nullable=False)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'),
        nullable=False)
    challenge_id = db.Column(db.Integer, db.ForeignKey('challenges.id',
        ondelete='CASCADE'), nullable=False)
    start_time = db.Column(db.DateTime, default=datetime.datetime.utcnow,
        nullable=False)
    end_time = db.Column(db.DateTime, nullable=True)
    duration = db.Column(db.Integer, default=0, nullable=False)

    def __init__(self, challenge_id, team_id, user_id):
        self.challenge_id = challenge_id
        self.team_id = team_id
        self.user_id = user_id # Initialisation de la clé utilisateur
        self.start_time = datetime.datetime.utcnow()
```

□ **user_id (Clé Étrangère) :** Connecte directement chaque segment de temps à la table native `users` de CTFd.

□ **Intégrité Référentielle (ondelete='CASCADE') :** Si un compte étudiant est purgé ou réinitialisé par le professeur dans l'onglet d'administration, MySQL propage automatiquement la suppression et nettoie toutes les lignes de télémétrie associée à cet ID, évitant ainsi le stockage de données orphelines.

3. Logique Métier Backend Révisée (Python)

L'intégration de l'identité de l'utilisateur a nécessité une mise à jour des hooks Flask pour capturer dynamiquement la session de l'étudiant actif.

A. Capture dynamique de l'identité :

Lorsqu'une interaction avec un exercice survient, le contrôleur backend n'interroge plus seulement l'équipe, il extrait l'objet utilisateur actuellement stocké dans la session de contexte sécurisée de CTFd via la fonction d'assistance native :

```
from CTFd.utils.user import get_current_user

user = get_current_user()
current_user_id = user.id
current_team_id = user.team_id
```

B. Isolation des Chronomètres Simultanés :

Grâce à cette modification, le backend gère parfaitement le cas où **deux membres d'une même équipe travaillent en même temps sur le même défi**.

- *Fonctionnement* : Puisque la clé primaire et l'unicité de la session d'écoute s'appuient désormais sur le couple (challenge_id, user_id), le système génère deux lignes de suivi indépendantes. L'étudiant A et l'étudiant B peuvent ouvrir le Défi A simultanément depuis leurs PC respectifs : le système comptabilisera le temps de chacun de manière isolée sans écraser les données du coéquipier.

C. Maintien de l'Agrégation de Groupe :

Pour préserver la logique d'équipe, les requêtes SQL du backend ont été adaptées. Le système peut toujours calculer le temps total passé par un groupe sur un exercice en regroupant les lignes par team_id, mais il offre maintenant la flexibilité d'isoler la contribution exacte de chaque individu en filtrant par user_id.

◆ Issue #5 — Performance Report per Team / per User (Matrice de Notation Enseignant)

1. Problématique et Objectif Fonctionnel

Dans un environnement de laboratoire ou d'examen académique, les enseignants doivent évaluer l'assimilation des compétences en cybersécurité de manière à la fois globale (par groupe) et nominative (par étudiant). Nativement, CTFd ne propose qu'un tableau des scores global basé uniquement sur le calcul des points accumulés, sans aucun indicateur d'efficacité ou de régularité. Un étudiant peut valider un défi après 50 tentatives infructueuses ou en copiant un flag, sans que l'enseignant ne puisse détecter ces difficultés à partir du classement général.

L'objectif de cette fonctionnalité lors du Sprint 2 était de concevoir une **Console de Suivi Pédagogique Centrale** (Dashboard Enseignant). Ce module agrège les tables de soumissions de flags et d'équipes pour calculer automatiquement des indicateurs de performance avancés : le taux d'exactitude (Accuracy Ratio), la répartition des réussites par catégorie thématique de

TP, et le volume d'erreurs commises, transformant des journaux bruts en une matrice de notation directement exploitable.

2. Algorithme d'Agrégation et Logique Backend (Python & SQLAlchemy)

La console enseignant n'utilise pas de table dédiée statique. Pour garantir la fraîcheur absolue des notes en direct, le contrôleur backend exécute des requêtes de regroupement SQL complexes à l'aide de l'ORM SQLAlchemy lors de chaque appel sur la route d'administration sécurisée /admin/performance/reports.

L'algorithme extrait et croise les données des tables natives de CTFd (users, teams, solves, submissions) selon deux matrices logiques distinctes :

A. Traitement et Calcul de la Matrice Étudiant (Per User) :

Pour chaque étudiant (user_id), le système calcule dynamiquement son profil d'efficacité :

1. **Nombre de défis résolus (Solves) :** Somme brute des entrées uniques dans la table solves pour cet identifiant.
2. **Nombre total de tentatives (Submissions) :** Nombre total de lignes (correctes et incorrectes) associées à l'étudiant dans la table submissions.
3. **Calcul de l'Accuracy Ratio (Taux d'Exactitude) :** Formule algorithmique calculant l'efficacité de résolution :
$$\text{Accuracy} = \left(\frac{\text{Nombre de Solves}}{\text{Nombre de Submissions}} \right) \times 100$$

Sécurité de division par zéro : Si un étudiant n'a soumis aucun flag, le système intercepte la condition et fixe l'Accuracy à 0.0% pour éviter un plantage de l'application.

B. Ventilation Catégorielle Thématique :

Pour chaque entité évaluée, le script parcourt les défis validés et comptabilise la répartition des points par étiquette de catégorie (Web, Crypto, Reverse, Forensic). Cela permet à l'enseignant de voir instantanément si un élève excelle en cryptographie mais échoue systématiquement sur les TP d'exploitation Web.

3. Interface Utilisateur & Intégration Graphique CSP-Safe (HTML & SVG)

L'affichage visuel des statistiques de performance au sein d'une application web pose des contraintes strictes. L'utilisation de bibliothèques graphiques JavaScript externes (comme Chart.js ou Google Charts) chargées depuis des CDN est interdite par la politique de sécurité native de CTFd (Content Security Policy - CSP) et ralentit l'affichage de la page lors du rendu de dizaines de profils.

Pour contourner cette limite technique, le frontend génère des **Graphiques Donut en pur format SVG injectés côté serveur (Server-Side Rendering) :**

```

<svg width="100" height="100" viewBox="0 0 42 42" class="donut-chart">
  <circle class="donut-hole" cx="21" cy="21" r="15.915" fill="#fff"></circle>
  <circle class="donut-ring" cx="21" cy="21" r="15.915" fill="transparent"
stroke="#e6e6e6" stroke-width="3"></circle>

  <circle class="donut-segment" cx="21" cy="21" r="15.915" fill="transparent"
stroke="#0056b3" stroke-width="3"
stroke-dasharray="{{ user_accuracy }}" {{ 100 - user_accuracy }}"
stroke-dashoffset="25"></circle>
</svg>

```

4. Filtrage et Structure Multidimensionnelle du Panneau Admin

L'interface d'administration offre une navigation structurée à double entrée grâce à un système d'onglets (Bootstrap Tabs) :

- **Onglet Général Utilisateurs** : Un grand tableau récapitulatif liste tous les étudiants inscrits de la salle de classe. Chaque ligne affiche le nom, l'équipe, le nombre de défis résolus, le nombre d'erreurs et le graphique SVG Donut représentant son taux d'exactitude.

◆ Issue #6 — Challenge Hint Usage Analytics (Tableau de Bord d'Analyse des Indices)

1. Problématique et Objectif Fonctionnel

Dans le cadre de laboratoires universitaires, les défis de cybersécurité intègrent souvent des indices (*hints*) pour guider les étudiants bloqués. Cependant, ces indices ont un coût pédagogique : ils réduisent le score maximal de l'exercice pour pénaliser l'aide. Nativement, CTFd enregistre l'achat d'un indice par une équipe, mais ne fournit aucune console statistique centralisée pour l'enseignant. Un professeur ne peut pas savoir si un exercice est trop difficile ou si la formulation d'un énoncé induit la classe en erreur.

L'objectif de cette fonctionnalité lors du Sprint 2 était de concevoir un **Module Pédagogique d'Analyse des Roadblocks (Blocages)**. Ce système agrège l'historique des déblocages d'indices pour fournir à l'enseignant des indicateurs précis : le taux d'utilisation des indices par défi, le coût total en points dépensé par la classe, et l'identification des exercices critiques qui freinent la progression, permettant ainsi d'ajuster le niveau des TP ou des cours.

2. Modélisation SQL et Logique d'Extraction Backend (Python & SQLAlchemy)

Le module d'analyse n'altère pas les structures d'origine. Il exploite les relations natives de CTFd en croisant trois tables du schéma via des requêtes de regroupement et des jointures relationnelles explicites :

1. **hints** : Contient la configuration de l'indice (ID du défi associé, texte explicatif, et coût en points).
2. **unlocks** : Journalise chaque transaction d'achat (ID de l'équipe, ID de l'indice débloqué, et date de l'action).

3. **challenges** : Fournit les métadonnées de l'exercice (nom et catégorie thématique).

Le contrôleur backend Python intercepte la route d'administration sécurisée /admin/performance/hints (protégée par @admins_only) et exécute un algorithme d'agrégation multiniveau :

```
from CTFd.models import db, Hints, Unlocks, Challenges

hint_stats = db.session.query(
    Challenges.id,
    Challenges.name,
    Challenges.category,
    db.func.count(Unlocks.id).label('total_unlocks'),
    db.func.sum(Hints.cost).label('total_points_spent')
).join(Hints, Hints.challenge_id == Challenges.id)\
.join(Unlocks, Unlocks.target == Hints.id)\
.filter(Unlocks.type == 'hints')\
.group_by(Challenges.id).all()
```

Analyse des indicateurs calculés par l'algorithme :

- **Volume Global de Déblocages (total_unlocks)** : Compte le nombre total de fois que les étudiants ont acheté une aide pour un défi spécifique. Un volume anormalement proche du nombre total d'élèves indique un verrou méthodologique majeur dans le TP.
- **Investissement de la Classe (total_points_spent)** : Calcule la somme cumulée des points sacrifiés par les étudiants pour obtenir de l'aide. Plus ce score est élevé, plus le défi a impacté négativement les notes de la classe.

3. Interface Utilisateur Administrateur & Cartographie des Obstacles

Le frontend Jinja2 transforme ces tableaux d'agrégation en un panneau de contrôle clair et structuré pour l'équipe enseignante :

- **Le Tableau des Roadblocks Pédagogiques** : Présente la liste des défis ayant nécessité au moins un indice. Les lignes sont triées dynamiquement par ordre décroissant de consommation (total_unlocks DESC) pour placer les exercices les plus problématiques en tête de liste.
- **Indicateurs Clés Flash (KPI Cards)** : Le haut de la page affiche des compteurs globaux :
 - Le défi le plus coûteux de la séance.
 - L'indice le plus acheté par l'ensemble de la classe.
 - La moyenne de points sacrifiés par étudiant sur le laboratoire en cours.

◆ Issue #9 — Perceived Challenge Difficulty Rating (Interface de Notation et Feedbacks)

1. Problématique et Objectif Fonctionnel

Dans un environnement d'apprentissage de la cybersécurité, les enseignants se heurtent souvent à un manque de retours directs et structurés de la part des étudiants concernant la qualité et la clarté des exercices pratiques. Un défi peut être résolu avec succès, mais avoir provoqué de la frustration en raison d'un énoncé ambigu ou d'une panne d'infrastructure de TP. Nativement, CTFd ne propose aucun mécanisme de collecte de commentaires ou d'évaluation de la difficulté ressentie par les participants.

L'objectif de cette fonctionnalité lors du Sprint 2 était de concevoir un **Module Centralisé de Retours Pédagogiques**. Ce système se divise en deux volets : une interface utilisateur côté étudiant pour recueillir à chaud une évaluation binaire (évaluation positive ou négative) et un commentaire textuel lors de la validation du flag, et une console d'administration côté enseignant pour centraliser et analyser ces retours sans perturber la mise en page.

2. Modélisation et Persistance des Données (Base de données MySQL)

Pour stocker de manière permanente l'évaluation de la difficulté ressentie et les commentaires des étudiants, une table relationnelle dédiée nommée `challenge_ratings` a été injectée dans l'instance MySQL à l'aide de l'ORM SQLAlchemy.

```
from CTFd.models import db
import datetime

class ChallengeRatings(db.Model):
    __tablename__ = 'challenge_ratings'
    id = db.Column(db.Integer, primary_key=True)
    challenge_id = db.Column(db.Integer, db.ForeignKey('challenges.id',
        ondelete='CASCADE'), nullable=False)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'),
        nullable=False)
    rating = db.Column(db.String(16), nullable=False) # 'upvote' ou 'downvote'
    comment = db.Column(db.Text, nullable=True)
    timestamp = db.Column(db.DateTime, default=datetime.datetime.utcnow, nullable=False)

    def __init__(self, challenge_id, user_id, rating, comment=None):
        self.challenge_id = challenge_id
        self.user_id = user_id
        self.rating = rating
        self.comment = comment
```

- ❑ **id (Clé Primaire)** : Identifiant unique auto-incrémenté pour chaque évaluation soumise.
- ❑ **challenge_id & user_id (Clés Étrangères)** : Associent chaque note à un exercice et à un étudiant unique, avec une contrainte de nettoyage en cascade (`ondelete='CASCADE'`) pour éviter les lignes orphelines.

□ **rating** : Chaîne de caractères limitant strictement la notation à un choix binaire prédéfini (upvote ou downvote).

□ **comment** : Champ textuel de type Text stockant les remarques libres de l'élève.

3. Logique Métier Backend et Capture des Avis (Python)

La logique backend s'articule autour de l'interception de la validation des défis et de l'exposition d'un point de terminaison (endpoint) dédié pour l'enregistrement sécurisé des avis.

A. Traitement et Enregistrement de la Note (POST /api/v1/challenges/rate) :

Une route API d'administration a été créée pour intercepter la soumission du formulaire d'évaluation. Ce contrôleur extrait l'ID de l'utilisateur actif de la session, valide l'intégrité des entrées pour s'assurer que l'étudiant a effectivement résolu le défi en question (vérification dans la table solves), puis insère ou met à jour la ligne correspondante dans challenge_ratings.

B. Algorithme de Calcul des Ratios pour l'Enseignant :

Lors de l'appel du panneau de gestion par l'enseignant, le backend n'interroge pas les lignes une par une. Il exécute une requête d'agrégation SQL cumulative pour calculer le pourcentage d'approbation globale de chaque exercice :

```
total_votes = total_upvotes + total_downvotes
approval_ratio = (total_upvotes / total_votes) * 100 if total_votes > 0 else 0
```

◆ Issue #8 — CSV / PDF Export of All Reports (Moteur d'Exportation Multi-Format)

1. Problématique et Objectif Fonctionnel

Dans le cadre de la gestion académique d'un laboratoire universitaire, la collecte de données sur la plateforme ne représente que la moitié du travail. Pour que ces informations de télémétrie (temps de travail, alertes de sécurité, utilisation des indices, retours pédagogiques) soient pleinement utiles, elles doivent pouvoir être extraites facilement du système. Les enseignants ont besoin de ces données pour alimenter leurs grilles de notation externes (comme Excel) ou pour archiver des rapports d'évaluation officiels au format papier ou numérique. Nativement, CTFd ne propose aucun système d'exportation ciblé pour les modules de télémétrie personnalisés.

L'objectif de cette fonctionnalité lors du Sprint 2 était de concevoir un **Moteur d'Extraction et de Formatage Multi-Format**. Ce sous-système offre deux solutions complémentaires : une extraction brute au format Excel CSV pour le traitement de données, et un moteur de mise en page PDF propre via les fonctionnalités d'impression natives, le tout accessible en un seul clic depuis les différents panneaux d'administration.

2. Architecture Backend de l'Extraction CSV (Python & Flask Streaming)

Pour permettre le téléchargement de volumes importants de lignes de logs sans surcharger la mémoire vive (RAM) du serveur Docker, le backend utilise un mécanisme de flux en streaming de données texte.

Des routes d'API d'administration dédiées (telles que `/admin/performance/export/csv`) ont été créées et protégées par le décorateur `@admins_only`. Le contrôleur Python utilise l'utilitaire `StringIO` et le module natif `csv` pour générer le fichier à la volée :

```
import csv
from io import StringIO
from flask import Response, stream_with_context
from CTFd.models import db, Challenges

def generate_csv_data(query_results, headers):
    output = StringIO()
    writer = csv.writer(output)
    |
    writer.writerow(headers)
    yield output.getvalue()
    output.seek(0)
    output.truncate(0)

    for row in query_results:
        writer.writerow(row)
        yield output.getvalue()
        output.seek(0)
        output.truncate(0)
```

Analyse du flux de transport de données :

Le contrôleur encapsule cette fonction de génération dans un wrapper Flask `Response(stream_with_context(...))`. Il configure les en-têtes HTTP de la réponse avec un type MIME spécifique (`text/csv`) et injecte la directive `Content-Disposition : attachment ; filename=rapport_performance.csv`. Le navigateur client intercepte ce flux et déclenche instantanément le téléchargement d'un **vrai fichier CSV Excel structuré**, garantissant une extraction de données propre et sans latence serveur

3. Moteur de Rendu et de Mise en Page PDF (@media print CSS Layers)

Pour générer des rapports PDF propres, l'utilisation de bibliothèques Python lourdes (comme `ReportLab` ou `WeasyPrint`) a été écartée car elles consomment trop de ressources CPU dans les conteneurs Docker et modifient souvent la mise en page des polices et des graphiques SVG.

À la place, une solution d'**Impression Vectorielle Native** a été développée en intégrant une couche de feuilles de style CSS spécialisée pour l'impression (`@media print`) dans tous nos templates HTML :

```

@media print {

    #sidebar, .navbar, .footer, .btn, .nav-tabs, .jumbotron {
        display: none !important;
    }
    body, .container, .card {
        background: #ffffff !important;
        color: #000000 !important;
        width: 100% !important;
        margin: 0 !important;
        padding: 0 !important;
        box-shadow: none !important;
        border: none !important;
    }
    table {
        page-break-inside: auto;
        width: 100% !important;
    }
    tr {
        page-break-inside: avoid;
        page-break-after: auto;
    }
    * {
        -webkit-print-color-adjust: exact !important;
        print-color-adjust: exact !important;
    }
}

```

Fonctionnement du rendu PDF :

Lorsqu'un enseignant se trouve sur une page d'analyse (Performances, Indices, Sécurité) et clique sur le bouton "Enregistrer le rapport en PDF" (qui appelle la fonction JavaScript `window.print()`), le navigateur applique instantanément cette couche CSS d'impression :

- Les menus latéraux, barres de navigation, boutons d'action et onglets de CTFd disparaissent totalement de la mise en page.
- Les conteneurs de données s'élargissent pour occuper 100 % de la largeur d'une feuille A4 blanche.
- Les directives `page-break-inside: avoid` empêchent le navigateur de couper une ligne de tableau ou un profil d'étudiant en deux au milieu d'une page.
- Les propriétés `print-color-adjust: exact` forcent le maintien des couleurs de notre jauge de réussite et de nos graphiques SVG Donut.

Le document final enregistré ou imprimé est ainsi un **vrai rapport académique épuré et professionnel**.

4. Sprint 3 : Autonomie Étudiante, Traçabilité de Sécurité et Automatisation Administrative

Le troisième sprint a finalisé l'écosystème en introduisant l'accès aux métriques pour les étudiants, un grand livre d'audit pour la sécurité des configurations, et un moteur d'automatisation pour accélérer le déploiement des laboratoires.

◆ Issue #5 (User-Side) — Performance Report per Team / per User (Scorecard Privée Étudiant)

1. Problématique et Objectif Fonctionnel

Lors du Sprint 2, la matrice des performances a été développée exclusivement pour le panneau d'administration des enseignants. Cependant, dans un cadre pédagogique universitaire moderne, l'auto-évaluation est un facteur clé de progression. Les étudiants ont besoin de voir leurs propres statistiques (taux d'exactitude, répartition des points par catégorie, volume d'erreurs) en temps réel pour comprendre leurs forces et lacunes.

Le défi technique majeur de cette ouverture de données résidait dans le cloisonnement et la sécurité : les étudiants ne devaient en aucun cas pouvoir interroger les performances de leurs camarades ou des autres équipes. L'objectif de cette fonctionnalité lors du Sprint 3 était de concevoir une **Route Utilisateur Sécurisée et Isoler (/performance/team)** générant un tableau de bord d'auto-évaluation personnalisé, interactif et conforme aux exigences de confidentialité.

2. Architecture de Sécurité Backend et Contrôle d'Accès (Python & Flask)

Pour empêcher toute tentative d'espionnage informatique ou d'extraction de données non autorisées (via modification des paramètres de requêtes URL par exemple), la route a été blindée directement au niveau du contrôleur backend Python.

```
from flask import Blueprint, render_template, abort
from CTFd.utils.user import get_current_user
from CTFd.utils.decorators import authed_only

plugin_bp = Blueprint('user_performance_report', __name__, template_folder='templates')

@plugin_bp.route('/performance/team', methods=['GET'])
@authed_only
def view_team_scorecard():
    user = get_current_user()
    if not user:
        return abort(403) # Accès interdit

    current_team_id = user.team_id
    current_user_id = user.id

    if not current_team_id:
        return render_template('user_team_report.html', error="Vous devez faire partie d'une équipe pour voir ce rapport.")
    current_team_id et current_user_id
```

Analyse des mécanismes de protection appliqués :

- **Décorateur `@authed_only`** : Intercepte instantanément les connexions anonymes pour exiger une session valide.
- **Bannissement des variables d'URL** : La route ne prend aucun argument variable (pas de `/performance/team/<int:id>`). L'ID de l'équipe et de l'utilisateur est extrait directement de l'objet de session chiffré du serveur (`get_current_user()`). Un étudiant n'a donc aucun moyen technique de modifier un chiffre dans sa barre d'adresse pour consulter les notes d'une autre équipe.

3. Interface Utilisateur Étudiante et Export PDF d'Auto-Évaluation

Le fichier de template dédié `user_team_report.html` a été conçu pour offrir une esthétique moderne et épurée (Bootstrap Cards), intégrée dans le thème utilisateur global de la plateforme :

- **Indicateurs de Performance Privés** : L'équipe visualise son rang actuel (calculé par rapport à sa progression), sa jauge de précision globale, le nombre de flags erronés soumis, et une barre de progression visuelle pour chaque catégorie thématique de TP.
- **Bouton d'Exportation PDF Propre** : Pour permettre aux étudiants de joindre leur bilan de compétences à leurs livrables académiques (rapports de TP ou projets), la page intègre une feuille de style d'impression `@media print` épurée. En cliquant sur "Imprimer mon bilan", le navigateur génère un document PDF propre, masquant automatiquement les barres de menu utilisateur de CTFd pour ne conserver qu'une fiche d'évaluation officielle et présentable.

◆ Issue #12 — Grand Livre d'Audit du Système (Admin Activity Log)

1. Problématique et Objectif Fonctionnel

Dans un environnement académique ou de compétition, la sécurité de la plateforme ne dépend pas uniquement des barrières imposées aux étudiants. Elle dépend aussi de la traçabilité des actions menées par l'équipe d'encadrement. Si un compte enseignant ou administrateur est compromis, ou si une erreur humaine survient (suppression accidentelle d'un défi, modification d'un barème de points, validation manuelle suspecte), le système natif de CTFd ne conserve aucun historique centralisé des requêtes de modification.

L'objectif de cette fonctionnalité lors du Sprint 3 était de concevoir un **Grand Livre d'Audit Système autonome**. Ce module intercepte, analyse et enregistre de manière permanente toutes les requêtes de mutation exécutées par les administrateurs, fournissant ainsi une traçabilité informatique absolue pour garantir l'intégrité de l'infrastructure de notation.

2. Modélisation et Persistance des Données (Base de données MySQL)

Pour stocker cet historique d'audit de manière persistante sans interférer avec les tables de production de CTFd, une table relationnelle dédiée nommée `admin_activity_logs` ont été injectée dans l'instance MySQL à l'aide de l'ORM SQLAlchemy.

```

from CTFd.models import db
import datetime

class AdminActivityLogs(db.Model):
    __tablename__ = 'admin_activity_logs'
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='SET NULL'),
        nullable=True)
    admin_name = db.Column(db.String(128), nullable=False)
    action_type = db.Column(db.String(16), nullable=False)
    target_url = db.Column(db.String(256), nullable=False)
    payload_size = db.Column(db.Integer, default=0, nullable=False)
    ip_address = db.Column(db.String(45), nullable=False)
    timestamp = db.Column(db.DateTime, default=datetime.datetime.utcnow, nullable=False)

    def __init__(self, user_id, admin_name, action_type, target_url, payload_size,
        ip_address):
        self.user_id = user_id
        self.admin_name = admin_name
        self.action_type = action_type
        self.target_url = target_url
        self.payload_size = payload_size
        self.ip_address = ip_address

```

- **id (Clé Primaire)** : Identifiant unique auto-incrémenté pour chaque entrée d'audit.
- **user_id (Clé Étrangère)** : Lié à la table native users. La contrainte applique la règle **ondelete='SET NULL'**. Si un compte administrateur est supprimé de la plateforme, la ligne de log d'audit n'est pas effacée (ce qui masquerait ses actions passées) ; la case devient simplement NULL tandis que le champ textuel permanent admin_name conserve l'identité de l'opérateur.
- **action_type & target_url** : Enregistrent le type de modification HTTP exécuté et la route ciblée (ex : /api/v1/challenges/5).
- **payload_size (Entier)** : Stocke la longueur brute (en caractères) des données envoyées au serveur.
- **ip_address** : Stocke l'adresse réseau de la station de travail de l'administrateur, calibrée à une taille de 45 caractères pour supporter pleinement les formats IPv4 et IPv6.

3. Logique Métier Backend, Interception et Calcul de Métriques (Python)

Le système déploie un intercepteur global branché directement sur le cycle de vie applicatif de Flask à l'aide du hook `@app.before_request`. Cette fonction s'exécute de manière non bloquante à chaque requête entrante.

Le script applique une barrière de tri pour éliminer les requêtes de simple lecture (GET) afin de ne pas saturer la base de données. Il n'isole que les requêtes de modification

4. Panneau d'Audit Administrateur, Exports et Impression PDF

L'interface d'administration `/admin/activity/logs` (accessible uniquement aux super-administrateurs) rassemble toutes ces données dans un tableau de bord épuré :

- **Scannabilité Visuelle Avancée** : Chaque type d'action possède un badge de couleur Bootstrap distinct (Rouge pour DELETE, Orange pour PATCH/PUT, Vert pour POST) pour permettre une lecture rapide et intuitive du fil d'actualité des modifications.
- **Bouton d'Exportation CSV Excel** : Intègre notre standard d'extraction en un clic, permettant de télécharger l'intégralité du grand livre d'audit au format `admin_activity_logs.csv` pour les archivages de sécurité officiels.
- **Mise en page PDF @media print** : La feuille de style d'impression masque instantanément les menus de navigation de CTFd pour restituer une liste d'audit propre et conforme, prête à être imprimée ou sauvegardée en PDF pour les livrables de fin d'année.

◆ Issue #4 — Auto-fill Default Values for Challenge Fields (Hub d'Autofill des Défis)

1. Problématique et Objectif Fonctionnel

Lors de la préparation d'une session de travaux pratiques (TP) ou d'un examen, l'équipe enseignante doit concevoir et téléverser des dizaines de défis de sécurité sur la plateforme CTFd. Nativement, chaque fois qu'un administrateur clique sur le bouton de création d'un défi (Standard ou Dynamique), CTFd ouvre une fenêtre contextuelle (*modal*) avec des formulaires entièrement vides. L'enseignant se retrouve contraint de retaper manuellement les mêmes informations redondantes (la catégorie du TP en cours, les barèmes de points par défaut, l'état initial masqué et la structure de l'énoncé en Markdown). Cette saisie manuelle et répétitive représente une perte de temps considérable et augmente le risque d'erreurs de frappe ou d'incohérences de configuration.

L'objectif de cette fonctionnalité lors du Sprint 3 était de concevoir un **Système d'Automatisation des Formulaires de Création**. Ce module permet aux enseignants de définir une fois pour toutes leurs préférences globales dans un panneau d'administration dédié sous le menu *Extensions*. Ensuite, un moteur d'injection asynchrone pré-remplit instantanément et intelligemment tous les champs requis dès qu'un formulaire de création est généré à l'écran.

2. Architecture de Stockage des Préférences Backend (Python & MySQL)

Pour conserver les préférences de l'enseignant de manière durable, le plugin utilise l'infrastructure clé-valeur de la table native **config** de CTFd, évitant ainsi la création de tables lourdes.

Le contrôleur Python intégré dans `CTFd/plugins/challenge_autofill/__init__.py` expose une route d'administration sécurisée `/admin/autofill/settings` (protégée par `@admins_only`). Lors d'une requête de sauvegarde POST, la logique backend capture et enregistre indépendamment les configurations pour les deux types de défis pris en charge :

- **Préférences Standard**: `autofill_std_category`, `autofill_std_value`, `autofill_std_state`, `autofill_std_template`.

- **Préférences Dynamiques:** `autofill_dyn_category`, `autofill_dyn_initial`, `autofill_dyn_minimum`, `autofill_dyn_decay`, `autofill_dyn_state`, `autofill_dyn_template`.

3. Logique Métier d'Injection Asynchrone Frontend (JavaScript & API MutationObserver)

La principale difficulté technique réside dans le fait que CTFd charge ses formulaires de création de défis de manière dynamique via des requêtes AJAX (fenêtres contextuelles Bootstrap), sans recharger la page. Un script JavaScript classique (de type `window.onload`) est donc totalement inefficace car il s'exécute avant que le formulaire n'existe dans le code de la page.

Pour contourner cet obstacle, nous avons développé un moteur d'injection frontend basé sur l'API native **MutationObserver** de JavaScript, inséré dynamiquement via le hook Flask applicatif `@app.after_request`

Analyse de la logique d'exécution asynchrone :

- **API MutationObserver :** L'observateur surveille en continu l'arbre du document (`document.body`). Dès qu'un enseignant clique sur "Create Challenge" et que CTFd injecte le code HTML du formulaire dans la page, l'observateur intercepte instantanément la modification et déclenche la fonction `runAutofillAutomation()`.
- **Détection contextuelle par visibilité (`offsetParent`) :** Pour savoir si l'enseignant crée un défi Standard ou Dynamique, le script vérifie la visibilité à l'écran de l'élément `initialInput` via la propriété `.offsetParent !== null`. Si l'élément est masqué ou absent, le script applique intelligemment les valeurs par défaut du mode Standard ; s'il est visible, il déploie la configuration du mode Dynamique.
- **Respect de la saisie utilisateur :** Le script vérifie systématiquement que la valeur actuelle d'un champ est vide (`value === ""`) avant d'injecter la configuration par défaut. Cela permet à l'enseignant de modifier manuellement un paramètre sans que l'outil d'autofill ne vienne écraser ses modifications.
- **Contrôle de l'éditeur de texte CodeMirror :** CTFd utilise la bibliothèque CodeMirror pour la saisie des descriptions en Markdown. Le script accède directement à l'instance de l'éditeur de texte via `.CodeMirror.setValue()` pour y injecter proprement une structure d'énoncé type (Template Markdown) pré-enregistrée.

4. Interface Utilisateur de Configuration (Extensions Hub)

Le fichier template `autofill_settings.html` fournit une page de contrôle épurée et moderne (Bootstrap Cards) accessible directement depuis le menu déroulant des extensions de la barre de navigation d'administration globale :

- **Carte de Configuration Standard (Bleue) :** Permet de définir le nom de la catégorie (ex: *Lab-Exploitation*), les points fixes, l'état (Visible/Caché) et de rédiger le squelette de description Markdown par défaut.

- **Carte de Configuration Dynamique (Grise) :** Permet de configurer de manière isolée la catégorie des défis dégressifs, le score initial, le score minimum, le coefficient de dégradation (Decay) et son propre modèle d'énoncé Markdown

5. Sprint 4 : Contrôle du Temps, Gestion en Bloc et Clôture du Projet

Ce dernier sprint a permis de finaliser le projet en ajoutant des outils de gestion rapide pour les enseignants, en rédigeant la documentation technique et en préparant le rapport final.

◆ Issue #10 — Planification du Gel du Classement (Scoreboard Freeze)

1. Problématique et Objectif Fonctionnel

Dans l'organisation de sessions d'examens ou de compétitions de cybersécurité, la dernière heure est une phase stratégique critique. Pour maximiser le suspense et empêcher les équipes de calquer leur comportement sur celui des leaders, il est d'usage de geler le tableau des scores public (*Scoreboard*). Les étudiants continuent de soumettre des flags et d'engranger des points de manière privée, mais le classement général n'évolue plus à l'écran. Nativement, CTFd intègre un champ de gel dans ses paramètres, mais sa configuration oblige l'enseignant à calculer manuellement des timestamps complexes et à naviguer dans des menus enfouis, créant un risque d'oubli majeur au milieu du stress de la gestion de la classe.

L'objectif de cette fonctionnalité lors du Sprint 4 était de concevoir un **Système d'Automatisation Temporelle Synchrone**. Intégré sous forme de raccourci d'extension, ce module permet à l'enseignant de planifier le gel à une minute précise depuis un calendrier local connecté à sa montre, tandis qu'un exécuteur en arrière-plan se charge de verrouiller les flux de données au millième de seconde près.

2. Architecture de Persistance et Utilisation de la Table Config (MySQL)

Pour garantir une parfaite stabilité et éviter de créer des processus lourds ou des threads d'arrière-plan complexes (comme des tâches Cron Linux) qui consommeraient de la mémoire dans le conteneur Docker, le plugin s'appuie sur la table native **config** de CTFd [Fandom].

Notre formulaire d'administration enregistre de manière transparente trois clés de configuration distinctes au sein de cette table de type clé-valeur :

1. **freeze** : La clé d'origine de CTFd. Lorsqu'elle contient un timestamp entier, le framework gèle automatiquement le classement. Notre plugin écrit directement dedans à l'échéance.
2. **scheduled_freeze_enabled** : Clé personnalisée créée à la volée (0 ou 1) pour mémoriser si le mode d'automatisation de l'extension est armé ou désactivé.
3. **scheduled_freeze_time** : Clé personnalisée stockant la chaîne de caractères brute de la date et de l'heure cible (format ISO).

3. Logique Métier d'Évaluation Globale Temporelle (Python & Cycle de Vie Flask)

La détection de l'échéance et l'exécution du gel reposent sur l'utilisation du hook de cycle de vie applicatif Flask `@app.before_request`. Ce choix technique permet de vérifier l'horloge système de manière transparente à chaque fois qu'un utilisateur ou un navigateur interagit avec la plateforme, assurant une précision absolue

Analyse de la synchronisation contre les dérives de fuseaux horaires :

Le plus grand défi de ce module réside dans le décalage horaire. Le serveur Docker fonctionne généralement à l'heure universelle UTC, tandis que l'ordinateur de l'enseignant est réglé sur l'heure locale.

Pour éliminer les bugs de calcul (qui gèleraient le classement instantanément ou plusieurs heures trop tard), le script backend utilise la fonction `calendar.timegm()`. Contrairement à `time.mktime()`, cette fonction traite le tuple de temps sans y appliquer le fuseau horaire de la machine hôte [Fandom]. Le calcul de l'entier Unix Epoch est donc parfaitement aligné sur l'horloge UTC interne de CTFd, garantissant un déclenchement exact à la seconde choisie.

4. Interface Graphique et Conversion Dynamique Frontend (HTML & JavaScript)

Le fichier template `freeze_settings.html` fournit une page de contrôle épurée et sécurisée. La saisie de la date utilise le type de champ standard `<input type="datetime-local">`, qui s'affiche automatiquement au format de la région de l'ordinateur de l'enseignant (Month/Day/Year ou Day/Month/Year).

Pour simplifier l'expérience utilisateur, une routine JavaScript (intégrée dans la balise `<script>`) a été développée pour gérer automatiquement les conversions de fuseaux horaires :

- **Au chargement de la page :** Si une date de gel est déjà stockée en base de données, le script JavaScript intercepte la chaîne UTC, calcule le décalage horaire de l'ordinateur (`getTimezoneOffset()`), et convertit la valeur pour l'afficher à l'heure locale exacte de l'enseignant sur le calendrier.
- **À la soumission du formulaire :** Dès que l'enseignant clique sur "Lock Scheduled Freeze Window", le script JavaScript intercepte l'événement, convertit instantanément la date locale choisie en une chaîne universelle normalisée. `toISOString()`, et l'envoie de manière transparente au backend Python.

L'enseignant manipule ainsi uniquement son heure locale (celle de sa montre) sans aucune formule de calcul mental, tandis que le système garantit une exécution technique universelle et sans faille en arrière-plan.

◆ Issue #15 — Modification de Plusieurs Défis à la Fois (Bulk Modifier)

1. Problématique et Objectif Fonctionnel

Lors de la gestion de sessions de travaux pratiques ou d'examens en direct, les enseignants doivent fréquemment ajuster les paramètres des exercices pour s'adapter au rythme de la classe (par exemple, réduire la valeur de plusieurs défis pour aider les étudiants en retard, ou masquer une catégorie entière d'exercices non abordés). Nativement, CTFd impose d'ouvrir chaque défi individuellement, de modifier ses champs, puis de sauvegarder. Cette manipulation est extrêmement chronophage et stressante en pleine séance.

L'objectif de cette fonctionnalité lors du Sprint 4 était de concevoir un **Contrôleur Polymorphique de Modification de Masse**. Intégré directement au bas du hub d'autofill, ce module permet à l'enseignant de sélectionner une catégorie entière et d'appliquer instantanément un nouveau barème de points ou un changement d'état (Visible/Masqué) à tous les défis correspondants en une seule action, tout en respectant les spécificités des défis standards et dynamiques.

2. Architecture de Filtrage et Logique Backend Segmentée (Python & SQLAlchemy)

Pour éviter qu'un enseignant n'écrase accidentellement les paramètres complexes d'un défi dynamique en voulant modifier un défi standard, le backend applique un **Découplage Strict par Type de Défi** au niveau du contrôleur Flask dans CTFd/plugins/challenge_autofill/__init__.py.

Lors de la soumission du formulaire de traitement de masse, le système sépare les requêtes en deux actions transactionnelles distinctes :

A. Bloc d'Action Standard (action_type == 'bulk_modify_standard') :

Le script isole uniquement les défis qui ne possèdent pas le comportement dynamique et applique les modifications directement sur la table principale

B. Bloc d'Action Dynamique (action_type == 'bulk_modify_dynamic') :

C'est ici que réside la complexité technique. Les défis dynamiques dégressifs stockent leurs scores de départ et planchers dans une table externe nommée `dynamic_challenge`. Si l'on modifie uniquement la table de base, CTFd écrase les changements au prochain rafraîchissement à cause de son cache interne.

Pour résoudre ce problème de synchronisation, l'algorithme utilise l'**Inspection d'Attributs par Réflexion (hasattr)** pour forcer la mise à jour simultanée des objets relationnels et vider le cache d'origine

3. Logique d'Extraction des Listes de Catégories Typées (Précision de l'Interface)

Pour garantir une interface sans erreurs, le contrôleur backend filtre l'extraction des listes de catégories transmises au fichier HTML. Au lieu de proposer un menu déroulant contenant toutes les catégories mélangées, le script exécute des requêtes de filtrage distinctes :

- **std_categories** : Extrait uniquement les noms de catégories uniques associés à des défis de type standard.
- **dyn_categories** : Extrait uniquement les noms de catégories uniques associés à des défis de type dynamic.

Cela empêche physiquement l'enseignant de sélectionner une catégorie contenant des défis dynamiques depuis l'interface d'action des défis standards, éliminant ainsi les fausses manipulations.

4. Interface Utilisateur Intégrée (Frontend HTML & CSS)

Le bas du template `autofill_settings.html` a été enrichi d'une section graphique distincte nommée *"Bulk Challenge Multi-Modifiers"*. Elle sépare visuellement le traitement en deux colonnes d'action claires (Bootstrap Grids) :

- **Colonne Bleue (Standard)** : Comprend le sélecteur filtré des catégories standards, un champ numérique pour modifier la valeur fixe et un menu déroulant pour forcer l'état de visibilité (Publish/Hide).
- **Colonne Grise (Dynamique)** : Comprend le sélecteur filtré des catégories dynamiques, deux zones de saisie numérique distinctes pour modifier simultanément le **Score Initial** et le **Score Minimum (Plancher)**, ainsi que le contrôle de visibilité.

Chaque bouton d'action intègre un déclencheur JavaScript d'avertissement de sécurité (onclick="return confirm(...);"). L'enseignant doit ainsi valider explicitement une fenêtre de confirmation avant que la requête ne soit transmise au serveur, protégeant la base de données contre les clics accidentels en cours d'examen.

◆ Documentation Technique du Code (How to use current repo)

- **Vision Complète** : Elle explique comment installer et comprendre le code créé pour que d'autres personnes puissent utiliser le projet facilement.
- **Réalisation** : Création d'un fichier de guide (documentation.md) placé à la racine du projet sur la branche **hamza**. Ce fichier liste l'emplacement des fichiers des 9 extensions ajoutées, explique la structure des nouvelles tables créées en base de données, et donne la commande Docker Compose pour lancer la plateforme en une seule ligne.

◆ Rédaction du Rapport Technique (Le présent document)

- **Vision Complète** : Synthétiser l'ensemble du travail accompli tout au long du stage dans un document clair et structuré.
- **Réalisation** : Rédaction de ce rapport d'activité (en cours de finalisation).

Partie 3 : Les Tables de la Base de Données

Pour faire fonctionner nos 9 outils, la plateforme utilise deux types de tables : les tables déjà présentes dans CTFd (pour lire les infos) et nos nouvelles tables (pour stocker nos mesures).

1. Les Tables d'Origine de CTFd (Comment nous les utilisons)

Nous utilisons ces tables pour lire ou modifier les informations de base de la plateforme :

- **Users** : Contient la liste des étudiants. Elle nous sert à savoir quel étudiant a fait quoi (pour le suivi du temps ou la sécurité).

- **Teams** : Regroupe les étudiants par équipe. Elle sert à afficher la page de score privée de chaque groupe (/performance/team).
 - **Challenges** : Contient tous les exercices (nom, catégorie, points). Notre outil d'autofill écrit dedans à la création, et l'outil de modification en bloc change les points ici.
 - **Dynamic_challenge** : Gère les exercices où les points baissent. Notre modificateur en bloc change les scores de départ et minimum ici.
 - **Solves** : Enregistre chaque fois qu'une équipe trouve la bonne réponse. Elle nous sert à calculer la moyenne de réussite de la classe.
 - **Submissions** : Garde l'historique de toutes les réponses tentées (bonnes et mauvaises). Notre sécurité l'analyse pour bloquer les étudiants qui devinent au hasard.
 - **Hints et unlocks** : Stockent les indices disponibles et qui les a achetés. Nous l'utilisons pour voir quels exercices bloquent les étudiants.
 - **Config** : Contient tous les réglages de CTFd. Notre bloqueur automatique (Scoreboard Freeze) écrit dedans pour masquer le classement à l'heure prévue.
-

2. Les Nouvelles Tables Créées (À quoi elles servent)

Nous avons créé ces tables pour enregistrer les nouvelles informations de nos extensions :

- **Sponsors** : Contient le nom et l'image des partenaires, ainsi qu'un numéro pour choisir l'ordre d'affichage sur la page d'accueil.
- **Challenge_time_track** : Note l'heure exacte du début et de la fin de travail d'un étudiant sur un exercice pour calculer son temps réel.
- **Challenge_clicks** : Compte combien de fois un étudiant ouvre ou ferme la fenêtre d'un exercice pour voir s'il change trop souvent de sujet.
- **Security_alerts** : C'est le carnet de notes de la triche. Si un étudiant devine trop vite ou partage ses réponses, son nom et l'alerte sont inscrits ici.
- **Classroom_open_logs** : Note l'heure précise où un étudiant ouvre un exercice pour la première fois, pour voir si la classe avance.
- **Admin_activity_logs** : Enregistre toutes les actions des profs (création de défis, suppression de comptes) avec leur nom, leur adresse IP et la taille du texte envoyé pour garder une trace de sécurité.

Partie 4 : Recommandations pour les Futurs Étudiants

Pour les prochains étudiants qui vont travailler sur cette plateforme ou continuer le projet sur la branche **hamza**, voici les meilleurs conseils, consignes de maintenance et pistes d'amélioration pour l'avenir :

💡 Conseils Techniques Essentiels

- **Comprendre l'architecture avant de coder :** Ne modifiez jamais le code source de base de CTFd [Fandom]. Utilisez toujours l'architecture des plugins (le dossier CTFd/plugins/) [Fandom]. Cela permet d'ajouter vos outils en toute sécurité sans casser le système d'origine [Fandom].
- **Faire attention aux types de données dans la base :** Lors des requêtes SQL, rappelez-vous que les défis standards et dynamiques sont stockés dans deux tables séparées [Fandom]. Utilisez toujours l'inspection d'attributs (hasattr) et faites attention à bien convertir les nombres en texte (str()) pour éviter les plantages (erreurs 500) [Shopify Community].
- **Tester avec les fuseaux horaires (Timezones) :** Le serveur Docker tourne souvent en heure universelle (UTC) alors que vos ordinateurs sont à l'heure locale [Fandom]. Utilisez toujours les fonctions universelles comme `calendar.timegm()` et `datetime.datetime.utcnow()` pour vos outils liés au temps (comme le minuteur ou le gel du classement) [Fandom].